

Experiment 1: CartPole

Deliverables

Two plots are provided on the next two pages. Both use `Eval_AverageReturn` versus `Train_EnvstepsSoFar`, as required.

Answers

1. Without advantage normalization, reward-to-go performed better than the trajectory-centric estimator. The small-batch ``cartpole_rtg`` run learned much faster and reached 200, while plain ``cartpole`` never reached 200 and finished at 49.78.

2. Reward-to-go is generally preferred because it reduces variance. It removes reward terms that occurred before action `a_t`, so the gradient estimate attributes credit more locally to the action that actually influenced those future rewards.

3. Advantage normalization helped a lot. In the small-batch case, both normalized runs (``cartpole_na`` and ``cartpole_rtg_na``) finished solved at 200. In the large-batch case, normalization also made convergence more reliable and prevented the late collapse seen in ``cartpole_lb``.

4. Batch size mattered. Larger batches made training smoother and let several unstable settings converge, but they also delayed feedback because each update required more interaction. Batch size improved stability, but variance-reduction tricks still mattered.

Experiment 1: Exact Commands Used

1. PYTHONPATH=src uv run python src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 --exp_name cartpole

2. PYTHONPATH=src uv run python src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 -rtg --exp_name cartpole_rtg

3. PYTHONPATH=src uv run python src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 -na --exp_name cartpole_na

4. PYTHONPATH=src uv run python src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 -rtg -na --exp_name cartpole_rtg_na

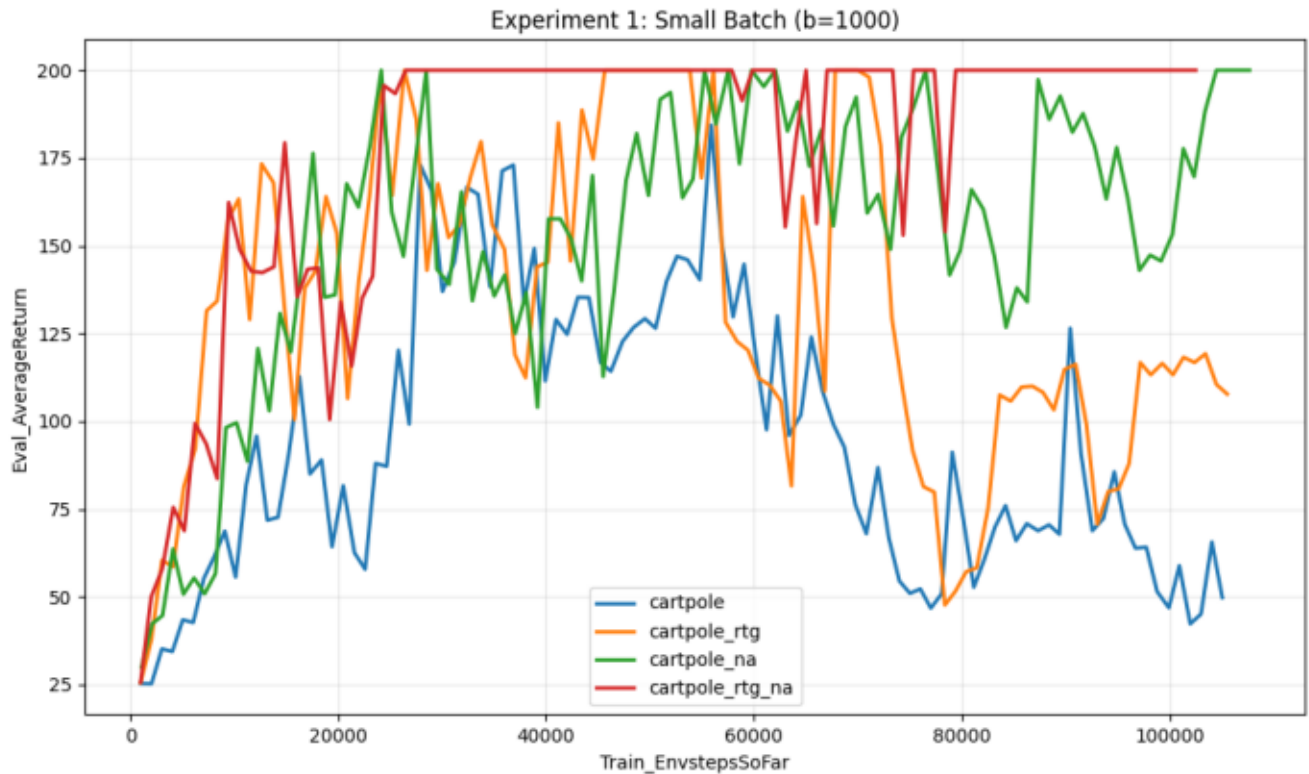
5. PYTHONPATH=src uv run python src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 --exp_name cartpole_lb

6. PYTHONPATH=src uv run python src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 -rtg --exp_name cartpole_lb_rtg

7. PYTHONPATH=src uv run python src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 -na --exp_name cartpole_lb_na

8. PYTHONPATH=src uv run python src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 -rtg -na --exp_name cartpole_lb_rtg_na

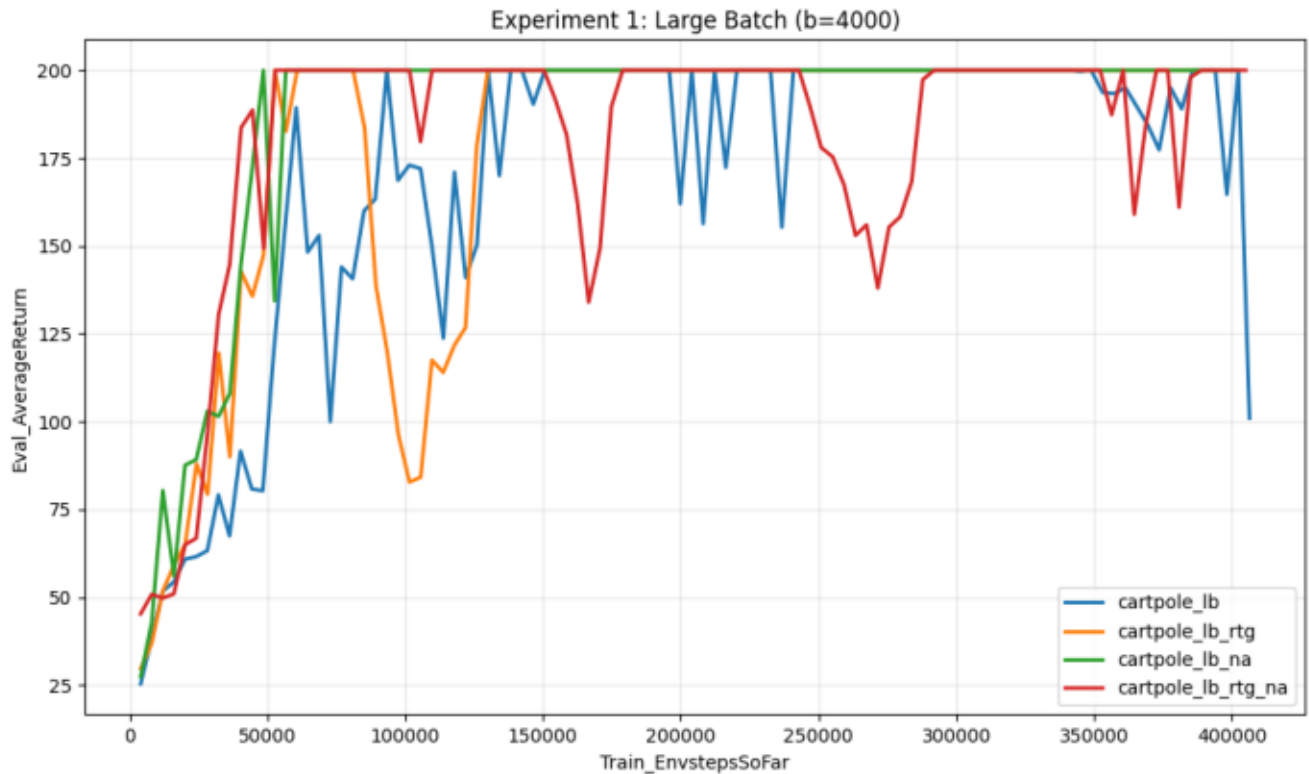
Experiment 1: Small-Batch Plot



Small-batch summary: `cartpole\_na` reached 200 first at 24,153 environment steps. `cartpole\_rtg\_na` also solved the task, reaching 200 at 26,513 steps. Without normalization, `cartpole\_rtg` still beat `cartpole`, but finished at 107.75 instead of staying solved.

`cartpole` peaked at 184.33 and never solved the environment, which supports the conclusion that reward-to-go and normalization both reduce gradient noise in this setting.

Experiment 1: Large-Batch Plot



Large-batch summary: `cartpole\_lb\_na`, `cartpole\_lb\_rtg`, and `cartpole\_lb\_rtg\_na` all finished at 200. The earliest of these was `cartpole\_lb\_na`, which first reached 200 at 48,546 steps. `cartpole\_lb` touched 200 once but collapsed and ended at 101.00.

`cartpole\_lb\_rtg` first reached 200 at 52,696 steps, and `cartpole\_lb\_rtg\_na` first reached 200 at 52,843 steps. Larger batches improved stability, but the best behavior still came from adding variance reduction.

Experiment 2: HalfCheetah

Deliverables

Required no-baseline run `cheetah`: final eval return -414.43, best eval return -265.67.

Required baseline run `cheetah\_baseline`: final eval return 355.87, best eval return 428.86.

`-na` no-baseline run `cheetah\_na`: final eval return 321.12, best eval return 336.29.

`-na` baseline run `cheetah\_baseline\_na`: final eval return 534.23, best eval return 534.23.

Decreased baseline-gradient-steps run `cheetah\_baseline\_bgs1\_na`: final eval return 239.27, best eval return 262.71.

Decreased baseline-learning-rate run `cheetah\_baseline\_blr005\_na`: final eval return 425.26, best eval return 490.73.

Answers

1. The baseline helped a lot. The original no-baseline run `cheetah` finished at -414.43, while the original baseline run `cheetah\_baseline` finished at 355.87 and peaked at 428.86.

2. Adding `-na` improved both the no-baseline and baseline settings. `cheetah\_na` finished at 321.12 versus -414.43 for `cheetah`, and `cheetah\_baseline\_na` finished at 534.23 versus 355.87 for `cheetah\_baseline`.

3. Decreasing `-bgs` from 5 to 1 hurt both the baseline fit and the final policy. `cheetah\_baseline\_bgs1\_na` ended at 239.27, well below `cheetah\_baseline\_na` at 534.23, and its baseline-loss curve stayed higher and noisier for longer.

4. Decreasing `-blr` from 0.01 to 0.005 still worked well here. `cheetah\_baseline\_blr005\_na` finished at 425.26 and peaked at 490.73. Its baseline-loss curve was generally tighter than the `-bgs 1` run, but still a bit worse than the best `cheetah\_baseline\_na` run.

Experiment 2: Exact Commands Used

1. PYTHONPATH=src uv run python src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg --discount 0.95 -lr 0.01 --exp_name cheetah

2. PYTHONPATH=src uv run python src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg --discount 0.95 -lr 0.01 --use_baseline -blr 0.01 -bgs 5 --exp_name cheetah_baseline

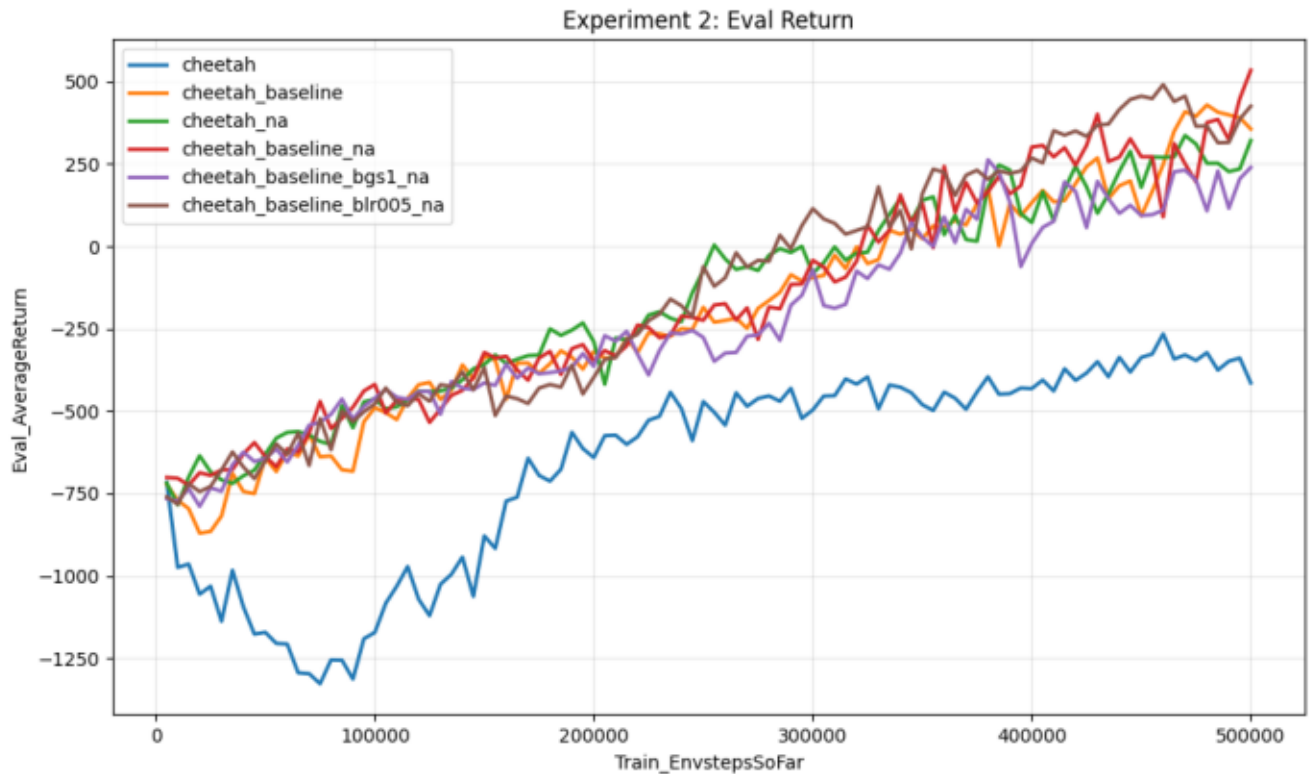
3. PYTHONPATH=src uv run python src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg --discount 0.95 -lr 0.01 -na --video_log_freq 10 --exp_name cheetah_na

4. PYTHONPATH=src uv run python src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg --discount 0.95 -lr 0.01 --use_baseline -blr 0.01 -bgs 5 -na --video_log_freq 10 --exp_name cheetah_baseline_na

5. PYTHONPATH=src uv run python src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg --discount 0.95 -lr 0.01 --use_baseline -blr 0.01 -bgs 1 -na --video_log_freq 10 --exp_name cheetah_baseline_bgs1_na

6. PYTHONPATH=src uv run python src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg --discount 0.95 -lr 0.01 --use_baseline -blr 0.005 -bgs 5 -na --video_log_freq 10 --exp_name cheetah_baseline_blr005_na

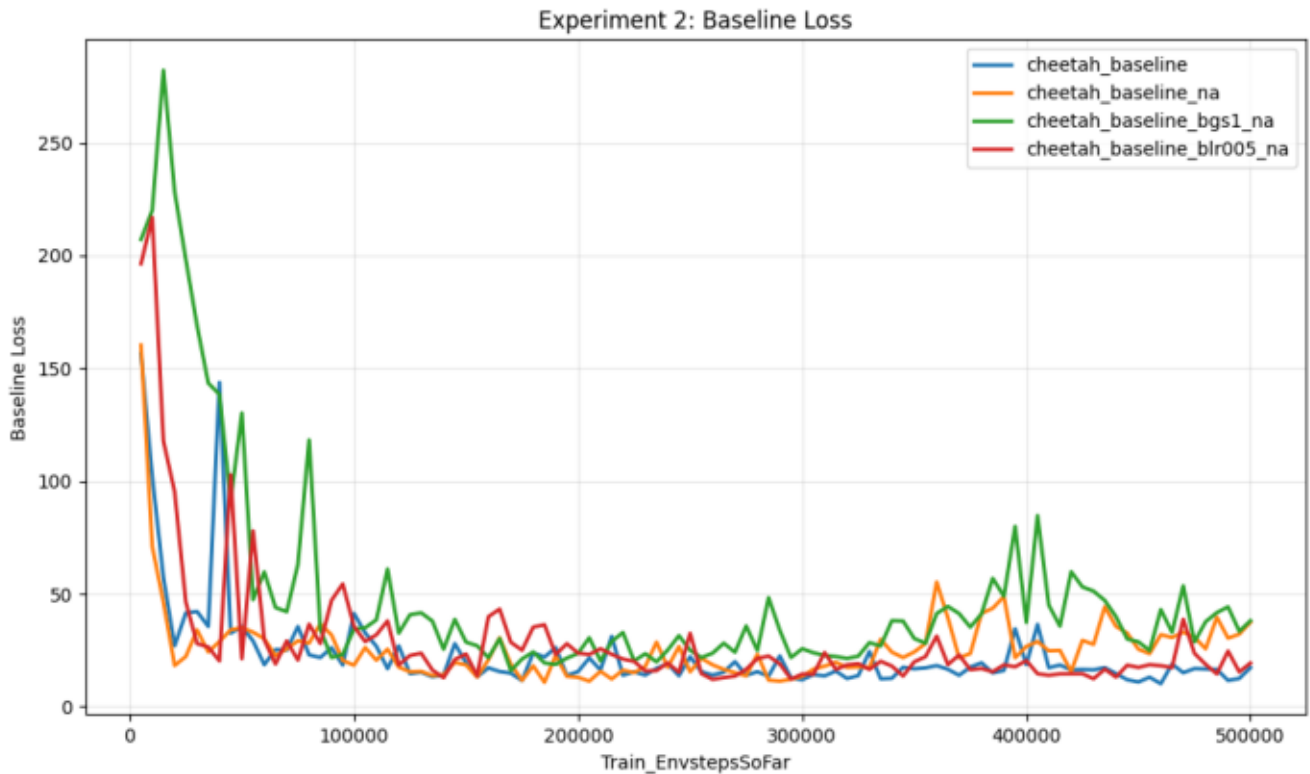
Experiment 2: Eval Return Plot



This plot compares the original required runs (`cheetah`, `cheetah\_baseline`) with the extra `-na` runs and the decreased-`bgs` / decreased-`blr` comparisons.

The main result is that `-na` helped a lot in HalfCheetah: `cheetah\_na` dramatically outperformed `cheetah`, and `cheetah\_baseline\_na` was the strongest overall configuration, finishing above 500 average eval return.

Experiment 2: Baseline Loss Plot



This plot shows the baseline loss for the baseline-enabled HalfCheetah runs. The ``-bgs 1`` run keeps a noticeably worse and noisier baseline fit than the full ``-bgs 5`` setting.

Comparing the ``-na`` variants, ``cheetah_baseline_na`` achieved the best final policy performance, while ``cheetah_baseline_blr005_na`` was a strong second. Both beat the weaker ``-bgs 1`` setting on policy return.